1.

Let Singly Linear Linked List delete be a function that deletes a node in a singly-linked list given a pointer to the node and a pointer to the head of the list. Similarly, let Doubly Linear LinkedList del be another function that deletes a node in a doubly-linked list given a pointer to the node and a pointer to the head of the list.

Let n denote the number of nodes in each of the linked lists. Which one of the following choices is TRUE about the worst-case time complexity of Singly Linear LinkedList del and Doubly Linear Linked List del?

A. SLL Del is O(1) and DLLdel is O(n)

B. Both SLLDel and DLLdel is O(log(n))

C. Both SLLDel and DLLdel isO(1)

D. SLLDel is O(n) and DLLdel is O(1)

Answer: D

2.

A queue is implemented using singly linear linked list. The queue has a head pointer and a tail pointer. enqueue to be implemented by inserting a newnode at the first position and a dequeue is implemented by deletion of a node from the last position.

Which of the following is the time complexity for the implementation of enqueue and dequeue respectively?

A. O(n), O(1)

B. O(n), O(n)

C. O(1), O(n)

D. O(1), O(1)

Answer: C

3.

The following C function takes a singly linear linked list as input argument. It modifies the list by moving the last element to the front of the list and returns the modified list. Some parts of the code are left blank.

```
typedef struct node{
    int value;
    struct node *next;
} Node;

Node* move_to_front(Node *head){
    Node *p, *q;
    if ((head == NULL) || (head->next == NULL))
        return head;
    q = NULL; p = head;
    while (p-> next !=NULL){
        q=p;
        p=p->next;
    }
    //blank line
    //blank line
    //blank line
return head;
}
```

Choose the correct alternative to replace the blank line.

A. q = null; p->next=head; q->next=NULL;

B. q->next = NULL; head=p; p->next=head;

C. head=p; p->next=q; q->next=NULL;

D. q->next=NULL; p->next=head; head=p;

Answer: D

4.

Consider the C code fragment given below:

```
typedef struct node
{
    int data;
    struct node *next;
}node_t;

void add(node_t *m, node_t *n)
{
    node_t *trav=n;
    while(trav->next!=NULL)
    {
        trav = trav->next;
    }
    trav->next=m;
}
```

Assuming that m and n point to valid NULL terminated linked list, invocation of join will

A. append list m to the end of list n for all inputs

B. either cause a null pointer dereference or append list m to the end of list n

C. cause a null pointer dereference for all inputs

D. append list n to the end of list m for all inputs

Answer: B

5.

The following sequence of operations are performed on the singly linked list

head -> A -->B -->C -->D-->E

Consider head is pointing to first node. Data of 5 nodes is A,B,C,D,E respectively.

```
struct node *temp;
temp=head->next->next;
temp->next->next->next = head;
printf("%d",temp->data);
```

The print statement will print _____ as output.

A. E

B. D

C. C

D. A

Answer:A